

Origin-Destination Matrix Estimation from SUMO Traffic with a Graph Neural Network

Study area: Tehran, District 2 — 36 TAZ zones, 1,897 road links

Abstract. We estimate a 36×36 origin-destination (OD) trip matrix from simulated traffic measurements on the District-2 road network — the inverse of the SUMO data generator in `od_generator/`. We show that the full OD matrix is **not identifiable** from aggregate link counts (an optimal linear inverse reaches only cell-correlation 0.14), but the **zone marginals are**. The key enabler is **junction turn counts**, recovered by re-simulating each scenario; with them a graph neural network predicts per-zone production/attraction at correlation ≈ 0.90 , and a doubly-constrained gravity (Furness) step reconstructs the matrix at **OD cell-correlation 0.78** with **total-flow error 13 %** — versus 0.06 / 44 % for a direct OD-regression baseline.

1. Objective

The forward pipeline samples a random OD matrix, routes it through SUMO, and records per-edge traffic. We build the **inverse**:

SUMO traffic measurements on District-2 links $\rightarrow$ **GNN** $\rightarrow$ 36×36 OD matrix.

The training set spans the full daily demand range (50–2,000 trips), grouped into four log-spaced demand tiers relabelled *night* $\rightarrow$ *noon* $\rightarrow$ *morning peak* $\rightarrow$ *evening peak* so every data split covers free-flow and congested regimes.

2. Data

2.1 Forward generator (existing)

`od_generator/` samples OD matrices from a mix of **gravity (40 %)**, **Dirichlet (35 %)** and **sparse-corridor (25 %)** processes, scales total demand across four log tiers, runs `od2trips` $\rightarrow$ `duarouter` $\rightarrow$ SUMO, and writes per-edge `flow`, `speed`, `density`, `traveltime` (plus two near-empty fields we drop). The dataset is **9,997 samples** (`od_dataset.h5`), over **36 TAZ zones** and **1,897 non-internal edges**. Every edge belongs to exactly one zone — a clean partition we exploit for pooling.

2.2 The identifiability problem

Recovering the full OD from aggregate link counts is the classical **under-determined** OD-estimation problem: many OD matrices produce the same link flows. A ridge regression from measurements to OD confirms it (1,500 train / 500 test):

input feature	OD cells (corr)	production (corr)	attraction (corr)
aggregate link flow (old)	0.06	0.45	0.28

input feature	OD cells (corr)	production (corr)	attraction (corr)
time-sliced link flow	0.05	0.36	0.25
junction turn counts	0.24	0.89	0.89

Link flow constrains the marginals only weakly and the cells not at all. **Turn counts** — how through-traffic splits at each junction — constrain routing and recover the marginals almost completely. From the *true* marginals, a gravity step reconstructs the OD at cell-correlation **0.85** (the ceiling); from turn-count-predicted marginals it already reaches **0.78**.

2.3 Turn-count enrichment (re-simulation)

`src/data/resimulate.py` re-runs SUMO 1.27 on the **existing** OD matrices and seeds (targets unchanged), capturing, per line-graph link, the **turn count** (vehicles traversing edge u then v). It is fast (~ 0.16 s/sample, 8 workers) and resumable. **5,000** scenarios were re-simulated for this report.

2.4 Enriched dataset

`src/data/build_enriched.py` produces, per sample, node features and per-link turn-count edge weights, split **3,500 / 750 / 750** (train/val/test), stratified by demand tier so all four regimes appear in every split. Normalisation ($\log_{1p}$ then standardise) is fit on the train split only.

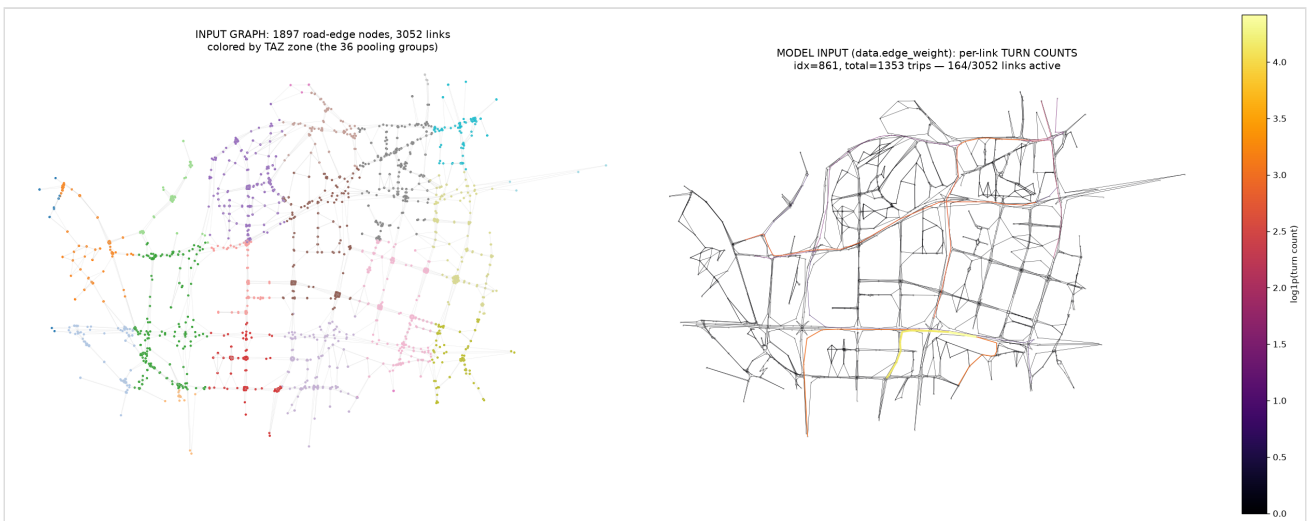


Figure 1. Input graph for one sample. Left: the 1,897 road-link nodes coloured by TAZ zone (the 36 pooling groups). Right: the actual model input — `data.edge_weight`, the per-link junction turn counts; bright / thick edges are heavily-used corridors.

3. Model

3.1 Input — the road *line-graph*

Each sample is a PyTorch-Geometric Data object:

component	shape	meaning
x (node features)	[1897, 9]	length, lanes, speed_limit, flow, speed, density, traveltime, out_turn, in_turn
edge_index	[2, 3052]	directed downstream connectivity between links
edge_weight	[3052]	per-link turn counts (message-passing weights)
zone	[1897]	TAZ membership (0-35) for pooling

Traffic enters both on **nodes** (flow/speed/density/traveltime) and on **edges** (turn counts).

3.2 Architecture — ODMarginalGNN

The network predicts **per-zone marginals**, not OD cells (the well-posed sub-problem):

1. **Zone embedding** Embedding(36 $\rightarrow$ 16), concatenated to node features.
2. **Encoder** MLP (9+16) $\rightarrow$ 128 $\rightarrow$ 128.
3. **4 $\times$ GraphConv(128)** message-passing layers, each consuming the turn-count edge_weight, with residual connection, LayerNorm, ReLU and dropout 0.1. Reverse edges are added so context flows both up- and downstream.
4. **Zone pooling**: mean | max | sum of node states within each TAZ $\rightarrow$ 36 zone vectors, projected to 128-d and LayerNorm-ed.
5. **Global context**: mean of the 36 zone vectors, concatenated to each.
6. **Two heads** (production, attraction): MLP 256 $\rightarrow$ 256 $\rightarrow$ 1, giving a per-zone scalar.

Total: $\approx$ **351k parameters**.

3.3 OD reconstruction — gravity / Furness

Predicted production P and attraction A are combined with a distance-decay deterrence $f(i,j) = \exp(-\beta \cdot d(i,j))$ on the zone grid, and balanced by iterative proportional fitting (src/od_reconstruct.py, $\beta = 1$, 40 iterations) so the reconstructed matrix matches the predicted marginals exactly while distributing trips by distance. This step is parameter-free to train.

4. Training

Target & loss. The decisive detail: marginals are fit as **per-zone standardised targets with MSE**. An earlier log1p+Huber objective regressed predictions toward the mean (std-ratio 0.35 — under-dispersed), which collapsed the gravity reconstruction. Standardised MSE restores the variance (std-ratio ≈ 1) and is what makes the marginals — and hence the OD — accurate.

Optimisation.

setting	value
optimiser	AdamW, lr 1e-3, weight decay 1e-4
schedule	cosine decay, 3 warm-up epochs
epochs / batch	25 / 64

setting	value
grad clip	1.0
conv / layers / hidden	GraphConv / 4 / 128
zone embed / zone hidden / readout hidden	16 / 128 / 256
dropout	0.1
device	CUDA (RTX 3060), ~7 s/epoch

Because this machine kills long GPU processes (~45 s watchdog), training checkpoints every epoch and **auto-resumes**, driven as short resumable chunks. Validation marginal correlation rises from ~0.40 (epoch 0) to **~0.90** by epoch 24.

5. Results

Held-out test set (750 samples). OD metrics are over off-diagonal cells, in trips.

approach	OD cell-corr	total-flow err	RMSE
direct-cell GNN (link counts)	0.06	0.44	6.78
marginal GNN + gravity (link counts)	0.44	0.49	6.78
turn-count GNN + gravity (this work)	0.78	0.13	4.08

Marginals. production corr **0.90** (MAE 4.7 trips), attraction corr **0.92** (MAE 4.5).

Per regime.

regime	MAE	RMSE	total-flow err	n
night	0.113	0.55	0.186	195
noon	0.265	1.17	0.130	184
morning peak	0.585	2.68	0.114	186
evening peak	1.339	7.66	0.097	185

MAE grows with demand (cells are larger), while *relative* total-flow error is ~10–19 % across all regimes.

6. Worked examples

Each figure shows the predicted OD heat-map, the true OD, per-zone outgoing demand (true vs predicted), and a per-cell scatter.

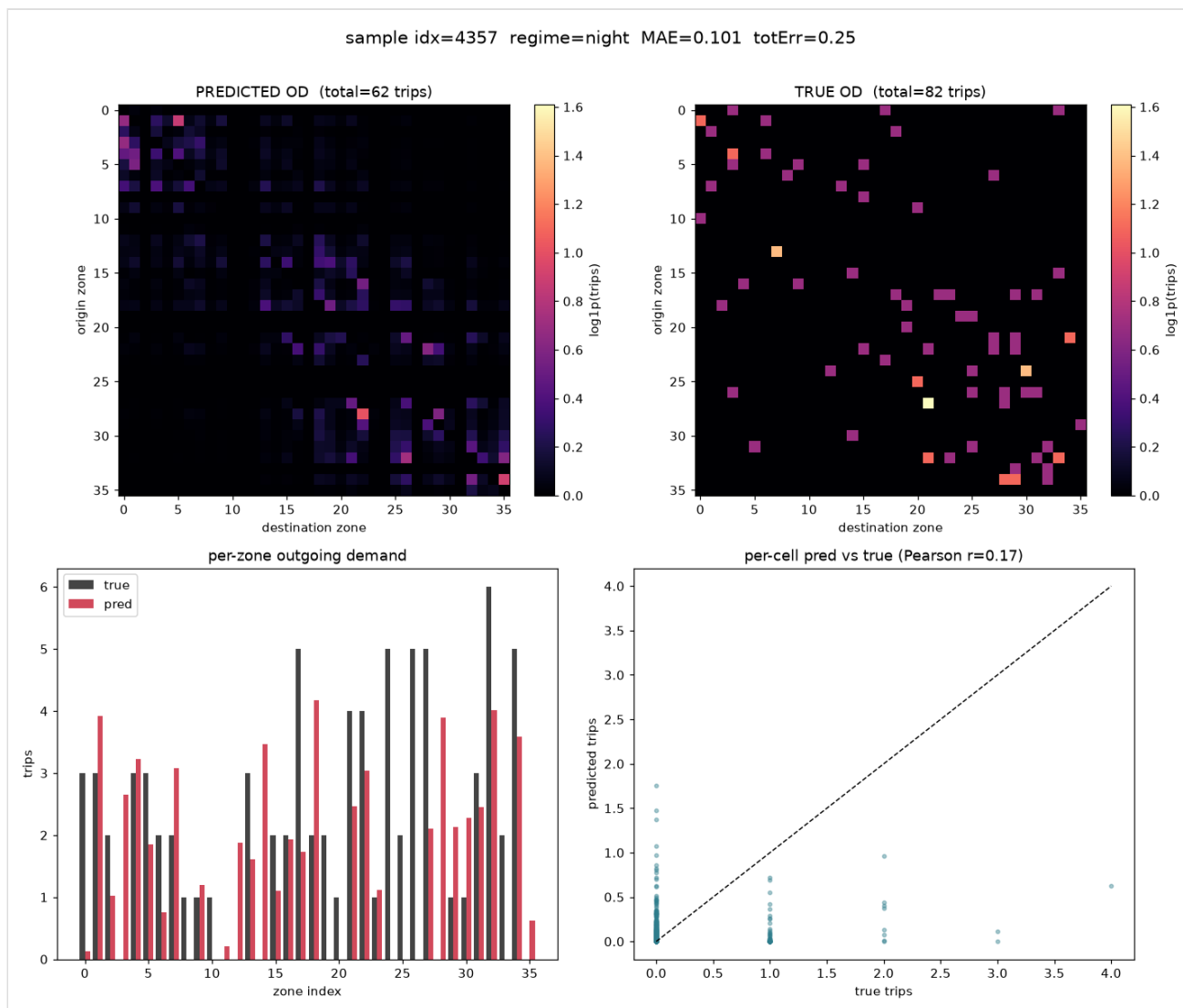


Figure 2. Example 1 — night, low demand (idx 4357, 82 trips). Predicted total 62; MAE 0.10. With few trips the signal is sparse; the model captures the demand level and rough spatial spread but not individual small pairs.

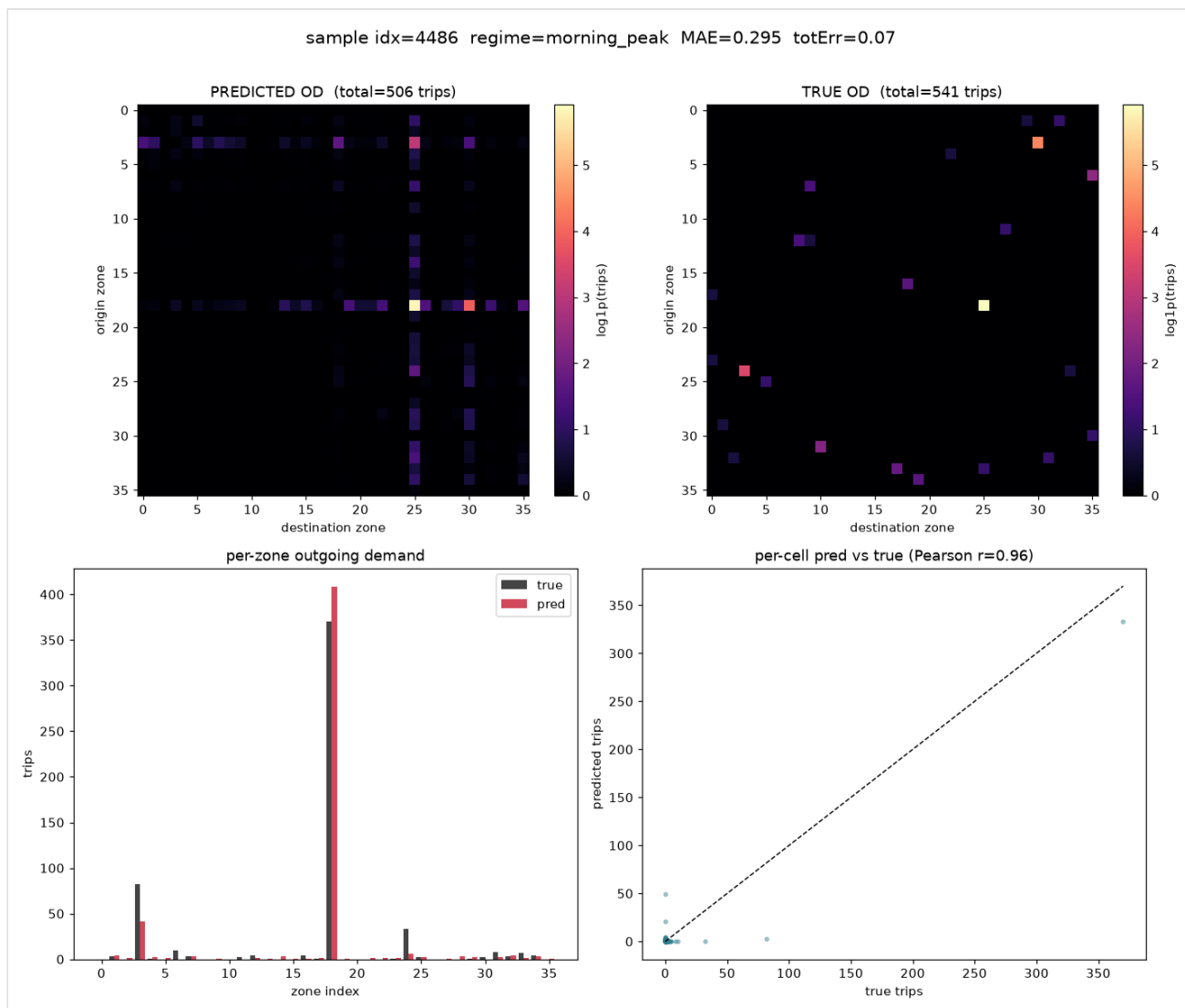


Figure 3. Example 2 — morning peak with a dominant corridor (idx 4486, 541 trips). Predicted total 506; MAE 0.30. The true matrix is dominated by 4_3→5_4 (370 trips); the model predicts the same top pair at 333 trips — the corridor is recovered in both location and magnitude.

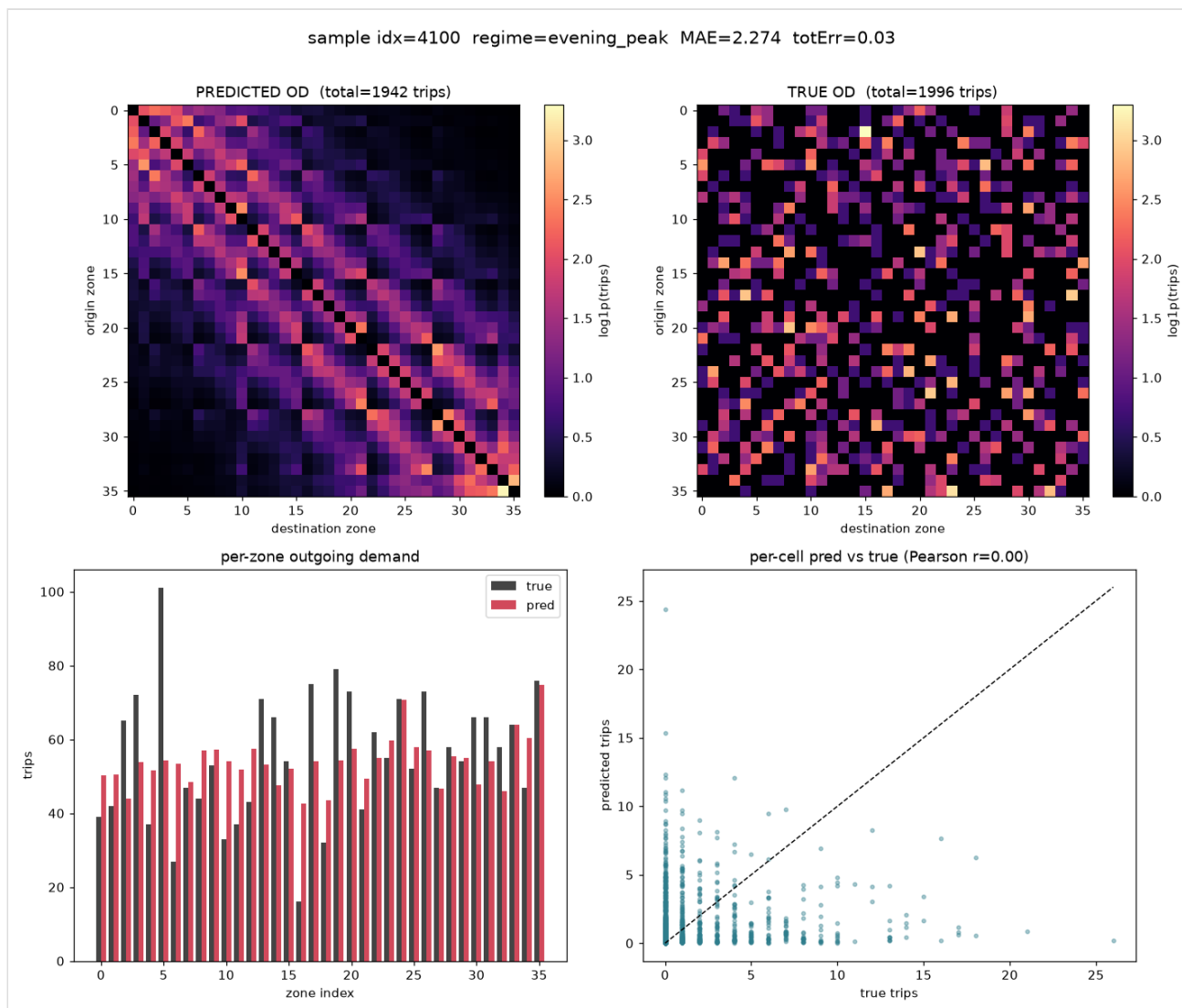


Figure 4. Example 3 — evening peak, dispersed demand (idx 4100, 1,996 trips). Predicted total 1,942 (within 3 %); marginals and the resulting traffic match closely, but individual cell identities differ — the residual under-determination for near-uniform OD.



Figure 5. Example 3, traffic. Traffic produced by the predicted OD (left) vs the true traffic (right), as junction turn counts on the network. They are almost identical (1,083 vs 973 active links) — many different ODs yield the same flows, which is exactly why the cell-level matrix stays partly unidentifiable.

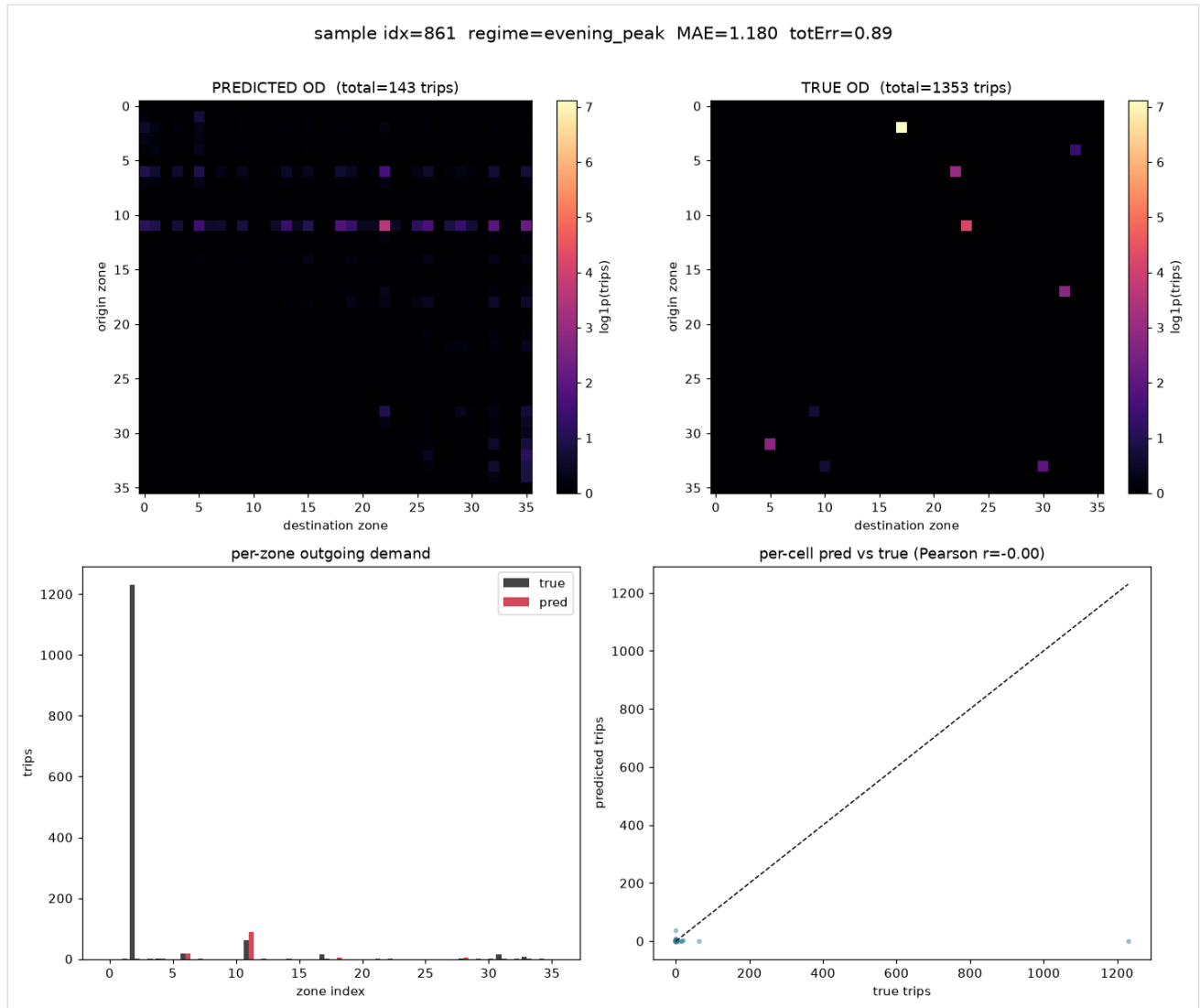


Figure 6. Example 4 — a failure case (idx 861, evening peak, 1,353 trips). A single 1,230-trip corridor $1_1 \rightarrow 4_2$ dominates; the model under-predicts the total badly (143) and misses it. Extreme single-corridor scenarios whose routing the model does not attribute to the origin zone remain hard; these outliers drive the high-demand RMSE.

7. Limitations and future work

- **Residual under-determination.** Dispersed ODs (Example 3) and extreme single-corridor ODs (Example 4) are partly or wholly unidentifiable even from turn counts, because different matrices generate the same observed flows. OD cell-corr 0.78 is near the 0.85 gravity ceiling.
- **Fixed gravity prior.** A single global distance-decay β is used; a learned or per-region deterrence should narrow the gap to the ceiling.
- **Partial re-simulation.** 5,000 of 9,997 scenarios were enriched; re-simulating the remainder (~ 15 min) would add training data.
- **Turn counts as input.** They assume junction movement counts are available (realistic from loop detectors / camera turning-movement counts).

8. Reproducibility

```
conda activate traffic
python -m src.data.resimulate --start 0 --end 5000 --workers 8 # capture turn counts
(resumable)
python -m src.data.build_enriched # -> data/
pack_enr_*.pt
python -m src.train # enriched marginal
GNN
python -m src.evaluate --save-preds # test metrics
python -m src.infer --index 4486 --plot-traffic # estimate OD +
figures
python -m src.plot_graph --index 4486 # input-graph figure
```

All hyperparameters live in `configs/{data,model,train}.yaml`; seeds are fixed; CUDA is auto-selected with CPU fallback.